Реимплементация DimExDAM: генерация 3D-микроструктур кернов по 2D-срезам

Описание кода, формулы и полный листинг

Unarov Aital

Октябрь 2026

Содержание

1 Обзор **3**
1.1 Структура проекта . . . 3
1.2 Поток данных . . . 3

2 Данные (`data.py`) **3**
2.1 Формат и нормировка . . . 3
2.2 Обучающие изображения . . . 4
2.3 Кропы и low-pass curriculum . . . 4
2.4 Стратифицированная нарезка 3D $\rightarrow$ 2D . . . 4

3 Диффузия (`diffusion.py`) **4**
3.1 Расписание шума . . . 4
3.2 Прямой процесс (реальные данные) . . . 4
3.3 Обратный шаг (генерация) . . . 5
3.4 Сэмплинг . . . 5

4 Модели (`models.py`) **5**
4.1 3D-генератор $G_\theta(x_{t+1}, t, z)$ . . . 5
4.2 2D-дискриминатор $D_\phi(x_t^{(p)}, x_{t+1}^{(p)}, t, p)$ . . . 6

5 Обучение (`train.py`) **6**
5.1 Лоссы . . . 6
5.2 Low-pass curriculum . . . 7
5.3 Итерация обучения . . . 7
5.4 Гиперпараметры . . . 7

6 Метрики (`metrics.py`) **7**

7 Скрипты **8**
7.1 `generate_samples.py` . . . 8
7.2 `calibrate_threshold.py` . . . 8
7.3 `view_volume.py` . . . 8

8 Результаты **8**
8.1 Ограничения . . . 9
8.2 Отличия от статьи и кода авторов . . . 9

A Листинг кода **10**
A.1 src/dimexdam/data.py 10
A.2 src/dimexdam/diffusion.py 11
A.3 src/dimexdam/models.py 12
A.4 src/dimexdam/train.py 13
A.5 src/dimexdam/metrics.py 16
A.6 scripts/generate_samples.py 16
A.7 scripts/calibrate_threshold.py 17
A.8 scripts/view_volume.py 19

1 Обзор

Проект реализует метод *DimExDAM* (Dimensionality Expansion Diffusion Adversarial Model) из статьи Aouf, Laloy, Rogiers, De Vleeschouwer, «Generating Heterogeneous 3D Geological Microstructures from 2D Images via a Stable Diffusion-Adversarial Model» (arXiv 2609.20358).

Задача. По трём ортогональным 2D-срезам бинарного керна (поры/твёрдая фаза) научиться генерировать новые 3D-объёмы, статистически эквивалентные исходному материалу. Модель безусловная: на каждый керн (вариант) обучается своя модель.

Идея метода. Гибрид диффузии и GAN (DDGAN): 3D-генератор за короткую цепочку из $T = 8$ шагов превращает шум в объём, а единственный 2D-дискриминатор на каждом шаге сравнивает пары срезов сгенерированного объёма с зашумлёнными реальными 2D-срезами. Вместо MSE-лосса денойзинга используется адверсариальный лосс, поэтому 3D ground truth не нужен.

Важно. Это собственная реимплементация по статье. Цикл обучения и часть гиперпараметров сверены с кодом авторов (`https://github.com/40uf411/DimExDAM`), но архитектура генератора упрощена (3D U-Net вместо NCSN++), а дискриминатор написан по описанию в статье. Отличия перечислены в разделе 8.2.

1.1 Структура проекта

Файл	Назначение
`src/dimexdam/data.py`	чтение `.mhd`, обучающие срезы, кропы, размытие, стратифицированная нарезка 3D→2D
`src/dimexdam/diffusion.py`	расписание шума, прямой процесс, апостериорное распределение, сэмплинг
`src/dimexdam/models.py`	3D-генератор (U-Net с AdaGN) и 2D-дискриминатор (spectral norm)
`src/dimexdam/train.py`	конфигурация, цикл обучения, сохранение/загрузка чекпойнтов
`src/dimexdam/metrics.py`	доля фазы, двухточечная функция $S_2(r)$, случайные подобъёмы
`scripts/generate_samples.py`	обучение (если нет чекпойнта) и генерация N объёмов, замеры времени и памяти
`scripts/calibrate_threshold.py`	порог бинаризации по пористости 2D-срезов, серые объёмы
`scripts/view_volume.py`	3D-рендер, листалка срезов, экспорт в `.mhd`
`notebooks/dimexdam_inference.ipynb`	ноутбук: обучение → инференс → сравнение с керном

1.2 Поток данных

1. Керн 300^3 (`data/Variant_XX/model.mhd`) → три центральных среза 300×300 (`ortho_slices`). Остальной объём модель не видит.
2. Обучение: случайные кропы 64×64 из срезов против стратифицированных срезов сгенерированных объёмов 64^3, на каждом шаге диффузии.
3. Инференс: шум → 8 проходов генератора → «серый» объём в $[-1, 1]$ → бинаризация (порог 0 или калиброванный).
4. Результат: `outputs/Variant_XX/samples.npy`, массив $(N, 64, 64, 64)$, `uint8`, 0 = поры, 1 = твёрдая фаза.

2 Данные (`data.py`)

2.1 Формат и нормировка

Керн хранится в формате MetaImage: заголовок `.mhd` и сжатые данные `.raw.gz`. Размеры в заголовке указаны в порядке $X\ Y\ Z$ (быстрее всего меняется X), поэтому массив читается как (Z, Y, X). Значения

бинарные, $v \in \{0, 1\}$, где 0 — поры. Для сети они переводятся в $\{-1, 1\}$:

$$x = 2v - 1, \qquad \text{поры} \mapsto -1, \quad \text{твёрдая фаза} \mapsto +1.$$

2.2 Обучающие изображения

Срез $p \in \{0, 1, 2\}$ перпендикулярен оси p и берётся в центре керна:

$$I_0 = V[150, :, :], \qquad I_1 = V[:, 150, :], \qquad I_2 = V[:, :, 150].$$

Это весь обучающий набор: по одному изображению 300×300 на плоскость, как в статье.

2.3 Кропы и low-pass curriculum

`SliceSampler` выдаёт n кропов $L \times L$ ($L = 64$): для каждого случайно выбирается плоскость p и положение кропа. Часть кропов размывается гауссовым ядром (curriculum, раздел 5.2). Ядро сепарабельное, с отражающим паддингом:

$$k_i = \frac{e^{-i^2/(2\sigma^2)}}{\sum_{j=-r}^{r} e^{-j^2/(2\sigma^2)}}, \qquad i = -r, \ldots, r, \quad r = \max(1, \lfloor 3\sigma \rfloor).$$

2.4 Стратифицированная нарезка 3D → 2D

Из каждого сгенерированного объёма b по каждой оси p вырезается S срезов. Индексы:

$$i_k = \left\lfloor \frac{(k + u_{b,p})\, L}{S} \right\rfloor, \qquad k = 0, \ldots, S-1, \quad u_{b,p} \sim \mathcal{U}[0, 1),$$

где сдвиг $u_{b,p}$ свой для каждого объёма и оси. Так срезы равномерно покрывают объём, а их положения меняются от итерации к итерации. Одни и те же индексы берутся из пары (x_t, x_{t+1}), потому что дискриминатор смотрит на согласованные пары.

3 Диффузия (`diffusion.py`)

3.1 Расписание шума

Косинусное расписание (Nichol & Dhariwal) с $T = 8$ и $s = 0{,}008$:

$$f(k) = \cos^2\!\left(\frac{k/T + s}{1 + s} \cdot \frac{\pi}{2}\right), \qquad \bar{\alpha}_k = \text{clip}\left(\frac{f(k)}{f(0)},\, 10^{-5},\, 1\right), \quad k = 0, \ldots, T,$$

$$\alpha_k = \bar{\alpha}_k / \bar{\alpha}_{k-1}, \qquad \beta_k = 1 - \alpha_k, \qquad \alpha_0 = 1.$$

k	0	1	2	3	4	5	6	7	8
$\bar{\alpha}_k$	1.000	0.958	0.847	0.684	0.494	0.304	0.144	0.038	10^{-5}
β_k	0	0.042	0.116	0.192	0.278	0.384	0.526	0.740	1.000

3.2 Прямой процесс (реальные данные)

Для реального кропа x_0 и шага $t \in \{0, \ldots, T-1\}$ строится пара (x_t, x_{t+1}) (`q_pair`):

$$x_t = \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1 - \bar{\alpha}_t}\, \varepsilon, \qquad x_{t+1} = \sqrt{\alpha_{t+1}}\, x_t + \sqrt{\beta_{t+1}}\, \varepsilon', \qquad \varepsilon, \varepsilon' \sim \mathcal{N}(0, I).$$

При $t = 0$ получается x_0 без шума ($\bar{\alpha}_0 = 1$).

3.3 Обратный шаг (генерация)

Генератор получает x_{t+1}, номер шага t и латентный вектор z и предсказывает чистый объём $\hat{x}_0 = G_\theta(x_{t+1}, t, z)$. Следующее состояние сэмплируется из апостериорного распределения DDPM (`posterior`):

$$q(x_t \mid x_{t+1}, \hat{x}_0) = \mathcal{N}\big(c_0(t)\,\hat{x}_0 + c_1(t)\,x_{t+1},\ \tilde{\sigma}_t^2 I\big)\,,$$

$$c_0(t) = \frac{\beta_{t+1}\sqrt{\bar{\alpha}_t}}{1-\bar{\alpha}_{t+1}}, \qquad c_1(t) = \frac{(1-\bar{\alpha}_t)\sqrt{\alpha_{t+1}}}{1-\bar{\alpha}_{t+1}}, \qquad \tilde{\sigma}_t^2 = \frac{\beta_{t+1}(1-\bar{\alpha}_t)}{1-\bar{\alpha}_{t+1}}.$$

На последнем шаге ($t = 0$) $\bar{\alpha}_0 = 1$, поэтому $c_0 = 1$, $c_1 = 0$, $\tilde{\sigma}_0 = 0$, и результат генерации в точности равен $\hat{x}_0 = \tanh(\ell)$, где ℓ — выход сети до tanh.

t	0	1	2	3	4	5	6	7
$c_0(t)$	1.000	0.740	0.560	0.455	0.388	0.339	0.292	0.194
$c_1(t)$	0	0.259	0.435	0.530	0.571	0.560	0.453	0.016
$\tilde{\sigma}_t$	0	0.179	0.305	0.417	0.528	0.654	0.811	0.981

В статье обратный шаг записан упрощённо: $x_{t-1} = \sqrt{\bar{\alpha}_{t-1}}\hat{x}_0 + \sqrt{1-\bar{\alpha}_{t-1}}\,\eta$. В коде авторов используется стандартное апостериорное распределение, как выше. Мы следуем коду.

3.4 Сэмплинг

Algorithm 1: Инференс (`Diffusion.sample`)

$x_T \sim \mathcal{N}(0, I)$ размера $n \times 1 \times L^3$; $\quad z \sim \mathcal{N}(0, I_{100})$ (один на всю траекторию);
for $t = T-1, \dots, 0$ **do**
  $\hat{x}_0 \leftarrow G_\theta(x_{t+1}, t, z)$;
  $x_t \leftarrow c_0(t)\hat{x}_0 + c_1(t)x_{t+1} + \mathbf{1}[t > 0]\,\tilde{\sigma}_t\,\varepsilon$;
end
return $x_0 \in [-1, 1]^{L^3}$ (при `logits=True` — логит ℓ последнего шага);

При `logits=True` апостериорный шаг всё равно вызывается: он расходует одно обращение к генератору случайных чисел, и следующие батчи получают тот же шум, что и при обычной генерации. Так результат с порогом 0 совпадает бит в бит.

4 Модели (`models.py`)

4.1 3D-генератор $G_\theta(x_{t+1}, t, z)$

Условие. Эмбеддинг шага и латентного вектора складываются:

$$e = E_t[t] + \mathrm{MLP}(z) \in \mathbb{R}^{128}, \qquad z \sim \mathcal{N}(0, I_{100}),$$

где E_t — обучаемая таблица эмбеддингов (T строк), а MLP состоит из трёх линейных слоёв с SiLU.

Блок ResBlock3D с AdaGN. Условие e модулирует нормализованные признаки:

$$h = \mathrm{Conv}_{3^3}\big(\mathrm{SiLU}(\mathrm{GN}(x))\big), \qquad [\gamma,\ \delta] = We + b,$$

$$\mathrm{out} = \mathrm{Conv}_{3^3}\Big(\mathrm{SiLU}\big(\mathrm{GN}(h) \odot (1+\gamma) + \delta\big)\Big) + \mathrm{skip}(x),$$

где GN — GroupNorm с 8 группами, а skip — свёртка 1^3 при смене числа каналов.

Архитектура (U-Net).

Уровень	Разрешение	Каналы	Операции
вход	64^3	$1 \to 32$	Conv 3^3
энкодер 1	64^3	32	ResBlock
энкодер 2	32^3	64	AvgPool, ResBlock
энкодер 3	16^3	128	AvgPool, ResBlock
энкодер 4	8^3	128	AvgPool, ResBlock
середина	8^3	128	ResBlock
декодер 1	16^3	$128 + 128 \to 128$	Upsample (nearest), concat, ResBlock
декодер 2	32^3	$128 + 64 \to 64$	Upsample, concat, ResBlock
декодер 3	64^3	$64 + 32 \to 32$	Upsample, concat, ResBlock
выход	64^3	$32 \to 1$	GN, SiLU, Conv $3^3 \to \ell$; $\hat{x}_0 = \tanh(\ell)$

Всего 4,83 M параметров. Сеть полностью свёрточная, поэтому на инференсе можно подавать объёмы любого размера, кратного 8. Но обучена она на 64^3, и на других размерах статистика смещается (раздел 8.1).

4.2 2D-дискриминатор $D_\phi(x_t^{(p)}, x_{t+1}^{(p)}, t, p)$

Один дискриминатор на все три плоскости. Вход — двухканальное изображение 64×64: пара срезов $[x_t^{(p)}, x_{t+1}^{(p)}]$. Условие:

$$e = \big[E_t[t];\ E_p[p]\big] \in \mathbb{R}^{256}.$$

Каждый слой получает условие как смещение по каналам:

$$h_l = \mathrm{LeakyReLU}_{0.2}\big(\mathrm{Conv}_l(h_{l-1}) + W_l e\big).$$

Свёртки: 3×3 (64 канала, 64^2), затем четыре свёртки 4×4 со страйдом 2: 128 (32^2), 256 (16^2), 512 (8^2), 512 (4^2). Голова — линейный слой $512 \cdot 16 \to 1$. Всего 7,34 M параметров.

Spectral normalization стоит на всех свёртках и линейных слоях:

$$W_{\mathrm{SN}} = \frac{W}{\sigma_{\max}(W)}.$$

$\sigma_{\max}$ оценивается степенным методом. Это ограничивает константу Липшица дискриминатора.

Выход. Логит обрезается и пропускается через tanh с температурой (раздел 3.3.1 статьи):

$$r = \mathrm{clip}(\mathrm{head}(h),\ -10,\ 10), \qquad D = \tanh(r/\tau), \quad \tau = 0{,}4.$$

Поэтому выход дискриминатора лежит в $[-1, 1]$, а минимум лосса D равен $2\,\mathrm{softplus}(-1) \approx 0{,}63$.

5 Обучение (`train.py`)

5.1 Лоссы

Используется non-saturating GAN loss. Реальные пары обозначены $x^{\mathrm{r}} = (x_t^{\mathrm{r}}, x_{t+1}^{\mathrm{r}})$, сгенерированные — $x^{\mathrm{g}} = (x_t^{\mathrm{g}}, x_{t+1}^{\mathrm{g}})$:

$$\mathcal{L}_D = \mathbb{E}\big[\mathrm{softplus}(-D(x^{\mathrm{r}}))\big] + \mathbb{E}\big[\mathrm{softplus}(D(x^{\mathrm{g}}))\big], \qquad \mathrm{softplus}(u) = \log(1 + e^u),$$

$$\mathcal{L}_G = \mathbb{E}\big[\mathrm{softplus}(-D(x^{\mathrm{g}}))\big].$$

Lazy R1. В каждой 16-й итерации (на всех 8 шагах t этой итерации) к $\mathcal{L}_D$ добавляется штраф на градиент по реальным входам:

$$\mathcal{L}_{R1} = \frac{\gamma}{2}\,\mathbb{E}\Big[\big\|\nabla_{x_t^{\mathrm{r}}} D(x_t^{\mathrm{r}}, x_{t+1}^{\mathrm{r}}, t, p)\big\|_2^2\Big], \qquad \gamma = 10.$$

Прочие стабилизаторы.

- TTUR: Adam с $\beta = (0;\ 0{,}9)$, $\eta_G = 2 \cdot 10^{-4}$, $\eta_D = 10^{-5}$.
- Клиппинг нормы градиента генератора: $\|\nabla_\theta\|_2 \le 3$.
- Клиппинг логитов D в $[-10, 10]$ и tanh с температурой (выше).
- bf16 autocast для прямых проходов. R1 считается в fp32.

5.2 Low-pass curriculum

Эпоха — 100 итераций (в статье размер эпохи не указан).

Итерации	σ	Доля размытых кропов
0–399 (эпохи 0–3)	0,8	65 %
400–699 (эпохи 4–6)	0,4	45 %
700 и далее	—	0

5.3 Итерация обучения

Порядок повторяет код авторов (`train_ddgan.py`): на каждом шаге цепочки сначала обновляется D, затем G, после чего состояние отсоединяется от графа. В Algorithm 1 статьи сначала прогоняется вся цепочка, затем делается одно обновление. Вариант авторского кода требует меньше памяти.

Algorithm 2: Одна итерация (`Trainer.train_iter`)

$x_T \sim \mathcal{N}(0, I)$, $B = 2$ объёма 64^3; $z \sim \mathcal{N}(0, I_{100})$;
for $t = T-1, \dots, 0$ **do**
  $x_t \leftarrow \text{posterior}\big(G_\theta(x_{t+1}, t, z),\, x_{t+1},\, t\big)$; `// с графом`
  фейковые пары: $S = 8$ стратифицированных срезов на ось из $(x_t, x_{t+1}) \Rightarrow 3 \cdot S \cdot B = 48$ пар;
  реальные пары: 32 кропа, размытие по curriculum, $(x_t^{\mathrm{r}}, x_{t+1}^{\mathrm{r}}) = \text{q_pair}(x_0^{\mathrm{r}}, t)$;
  шаг D: $\mathcal{L}_D$ (фейки отсоединены) + $\mathcal{L}_{R1}$, если итерация кратна 16; Adam;
  шаг G: $\mathcal{L}_G$ по тем же фейковым парам; клиппинг градиента; Adam;
  $x_{t+1} \leftarrow \text{stopgrad}(x_t)$;
end

5.4 Гиперпараметры

Параметр	Значение	Источник
Размер объёма / кропа	64^3 / 64^2	статья, код авторов
Шагов диффузии T	8	статья
Расписание	косинусное, $s = 0{,}008$	статья
Размерность z	100	код авторов
Объёмов на итерацию B	2	код авторов
Срезов на ось S	8	наш выбор (у авторов 16)
Реальных кропов на шаг	32	наш выбор
η_G / η_D	$2 \cdot 10^{-4}$ / 10^{-5}	код авторов
Adam β	$(0;\ 0{,}9)$	код авторов
R1: γ, период	10, каждые 16 итераций	статья
Клиппинг градиента G	3	статья
τ выхода D	0,4	статья
Итераций	3000 (24 000 обновлений D и G)	наш выбор
Seed обучения / генерации	0 / 1	—

6 Метрики (`metrics.py`)

Доля фазы (пористость). Для бинарного объёма V из N вокселей:

$$\phi = \frac{1}{N} \sum_{\mathbf{x}} \mathbf{1}[V(\mathbf{x}) = 0].$$

Двухточечная функция. Вероятность того, что обе точки на расстоянии r вдоль оси a лежат в порах. Для индикатора $\chi(\mathbf{x}) = \mathbf{1}[V(\mathbf{x}) = 0]$:

$$S_2^{(a)}(r) = \big\langle \chi(\mathbf{x})\, \chi(\mathbf{x} + r\mathbf{e}_a) \big\rangle_{\mathbf{x}}.$$

Считается через теорему Винера–Хинчина (периодическая граница) и усредняется по двум другим осям:

$$S_2^{(a)}(r) = \Big\langle \frac{1}{L}\, \mathcal{F}_a^{-1}\big[\, |\mathcal{F}_a[\chi]|^2 \,\big](r) \Big\rangle, \qquad r = 0, \ldots, 31.$$

Свойства: $S_2(0) = \phi$, а при больших r функция стремится к ϕ^2.

Эталон. `random_subvolumes` вырезает случайные кубы 64^3 из керна для сравнения со сгенерированными объёмами того же размера.

7 Скрипты

7.1 `generate_samples.py`

Для каждого варианта: если нет `checkpoints/Variant_XX.pt`, обучает модель (3000 итераций, seed 0). Затем генерирует N объёмов (seed 1, батчи по 4), бинаризует по порогу 0 и сохраняет `samples.npy`, картинку со срезами и `stats.json` (время и пиковую память GPU).

7.2 `calibrate_threshold.py`

Повторяет генерацию с тем же seed и проверяет, что порог 0 даёт в точности ранее сохранённый файл. Затем подбирает порог по пористости *2D-обучающих срезов*, то есть по информации, доступной методу:

$$\phi_{2D} = \frac{1}{3}\sum_{p=0}^{2} \frac{1}{|I_p|} \sum_{\mathbf{u}} \mathbf{1}[I_p(\mathbf{u}) = -1], \qquad c = Q_{\phi_{2D}}\big(\{\ell_i(\mathbf{x})\}_{i=1..N,\,\mathbf{x}}\big),$$

где Q_q — эмпирический q-квантиль логитов по всем N объёмам. Порог один на вариант, поэтому разброс между объёмами не подгоняется. Поры: $\ell < c$. Так как tanh монотонна, это равносильно порогу $\tanh(c)$ на сером выходе. Но около 99,5 % вокселей насыщены до ± 1, и порог по серому значению упирается в потолок, а по логиту — нет. Дополнительно скрипт считает пористость на гранях куба, чтобы контролировать краевой артефакт.

7.3 `view_volume.py`

Открывает `.npy` или `.mhd`. Режимы: 3D-изоповерхность выбранной фазы (marching cubes по уровню 0,5 после паддинга маски на 1 воксель, чтобы поверхность замыкалась на гранях, рендер в plotly), листалка срезов (matplotlib) и экспорт в `.mhd`/`.raw.gz`.

8 Результаты

20 объёмов 64^3 на вариант, RTX 5060 Ti 16 ГБ. ϕ_{2D} — пористость трёх обучающих срезов.

Вариант	ϕ керна	ϕ_{2D}	ϕ, порог 0	ϕ, калибровка	Обучение, мин	Генерация, с/объём
V01	0.429	0.426	0.418 ± 0.003	0.426 ± 0.003	$\approx 47^*$	0.19
V11	0.141	0.151	0.092 ± 0.006	0.151 ± 0.012	61.4	0.16
V02	0.197	0.202	0.130 ± 0.006	0.202 ± 0.008	59.4	0.16
V12	0.246	0.264	0.240 ± 0.005	0.264 ± 0.005	$\approx 81^*$	0.16
V18	0.340	0.349	0.335 ± 0.007	0.349 ± 0.008	60.0	0.16

* оценка по времени изменения файлов, замеры добавлены позже. Пиковая память (PyTorch allocated): обучение $\approx$ 2,5 ГБ, генерация $\approx 1{,}75$ ГБ.

8.1 Ограничения

- **Заниженный разброс.** Стандартное отклонение пористости между объёмами в ~ 10 раз меньше, чем у случайных кубов 64^3 керна (V01: 0,003 против 0,034). Авторы отмечают то же.
- **Тесные породы.** На V11 и V02 при пороге 0 пористость занижена примерно на треть, проводимость сильно занижена (по измерениям коллеги). Калибровка порога исправляет пористость.
- **Краевой артефакт.** На гранях объёма пористость выше, чем внутри: после калибровки для V11 крайний слой 0,23 против 0,15 внутри, для V02 0,30 против 0,20. У V12 то же уже при пороге 0. Генерация 80^3 с вырезанием центра убирает артефакт, но сдвигает статистику (модель обучена на 64^3).
- **Воспроизводимость.** Генерация из чекпойнта воспроизводится бит в бит. Обучение — нет: свёртки на GPU недетерминированы.

8.2 Отличия от статьи и кода авторов

- Генератор: компактный 3D U-Net с AdaGN (4,83 M) вместо NCSN++ авторов.
- Дискриминатор написан по описанию в статье. tanh с температурой на выходе D взят из текста статьи, в коде авторов его нет.
- Порядок обновлений и апостериорное распределение — как в коде авторов, а не как в Algorithm 1 и формуле статьи.
- R1, клиппинг градиента и размытие — по статье (в коде авторов по умолчанию другие значения: R1 выключен, клиппинг 12, размытие выключено).
- Размер эпохи (100 итераций), $S = 8$, 32 реальных кропа и 3000 итераций — наш выбор.
- Обучающие данные: по одному центральному срезу на плоскость, как в статье (в конфигурации кода авторов по 5 срезов на ось).

A Листинг кода

A.1 src/dimexdam/data.py

```python
"""Загрузка кернов (.mhd + .raw.gz) и нарезка 2D-срезов для обучения."""
import gzip
from pathlib import Path

import numpy as np
import torch
import torch.nn.functional as F


def load_mhd(path) -> np.ndarray:
    """Читает MetaImage. Возвращает массив uint8 формы (Z, Y, X)."""
    path = Path(path)
    hdr = dict(l.split(" = ", 1) for l in path.read_text().splitlines() if " = " in l)
    dims = [int(d) for d in hdr["DimSize"].split()]          # X Y Z (X меняется быстрее всего)
    raw = path.parent / hdr["ElementDataFile"]
    buf = gzip.open(raw).read() if raw.suffix == ".gz" else raw.read_bytes()
    return np.frombuffer(buf, np.uint8).reshape(dims[::-1])


def ortho_slices(vol: np.ndarray, idx=None) -> list[torch.Tensor]:
    """Три ортогональных среза: срез p перпендикулярен оси p. Это единственные данные для
    ↪ обучения
    (как в статье: по одному изображению на плоскость). Значения переводятся из {0,1} в
    ↪ {-1,1}."""
    idx = idx or [s // 2 for s in vol.shape]
    sl = [vol[idx[0]], vol[:, idx[1]], vol[:, :, idx[2]]]
    return [torch.from_numpy(s.astype(np.float32) * 2 - 1) for s in sl]


def gauss_blur(x: torch.Tensor, sigma: float) -> torch.Tensor:
    """Гауссово размытие батча (N,1,H,W): нужно для low-pass curriculum."""
    r = max(1, int(3 * sigma))
    k = torch.exp(-torch.arange(-r, r + 1, device=x.device) ** 2 / (2 * sigma**2))
    k = (k / k.sum()).view(1, 1, -1)
    x = F.conv2d(F.pad(x, (r, r, 0, 0), mode="reflect"), k[..., None, :])
    return F.conv2d(F.pad(x, (0, 0, r, r), mode="reflect"), k[..., :, None])


class SliceSampler:
    """Случайные кропы L×L из трёх обучающих срезов вместе с меткой плоскости."""

    def __init__(self, slices, crop: int, device):
        self.imgs = [s.to(device) for s in slices]
        self.L, self.device = crop, device

    def __call__(self, n: int, blur=(0.0, 0.0)):
        planes = torch.randint(0, 3, (n,))
        crops = []
        for p in planes.tolist():
            img = self.imgs[p]
            i, j = (np.random.randint(0, s - self.L + 1) for s in img.shape)
            crops.append(img[i:i + self.L, j:j + self.L])
        x = torch.stack(crops)[:, None]
        sigma, prob = blur                                   # curriculum: размываем долю prob
        ↪ кропов
        if sigma > 0:
```

```
            m = torch.rand(n, device=self.device) < prob
            if m.any():
                x[m] = gauss_blur(x[m], sigma)
        return x, planes.to(self.device)


def stratified_slices(a: torch.Tensor, b: torch.Tensor, S: int):
    """Вырезает одни и те же стратифицированные срезы из двух объёмов (x_t, x_{t+1}) по всем 3 осям.
    i_k = floor((k + rand) * L / S), где rand выбирается отдельно для каждой оси и каждого объёма."""
    B, L = a.shape[0], a.shape[-1]
    outs_a, outs_b, planes = [], [], []
    for p in range(3):
        idx = ((torch.arange(S, device=a.device) + torch.rand(B, 1, device=a.device)) * L / S).long()
        bi = torch.arange(B, device=a.device)[:, None]
        for v, out in ((a, outs_a), (b, outs_b)):
            out.append(v[:, 0].movedim(1 + p, 1)[bi, idx].reshape(B * S, 1, L, L))
        planes.append(torch.full((B * S,), p, device=a.device))
    return torch.cat(outs_a), torch.cat(outs_b), torch.cat(planes)
```

A.2 src/dimexdam/diffusion.py

```
"""Короткая диффузионная цепочка (T=8, косинусное расписание), как в DDGAN.

Индексация: x_0 — чистые данные, x_T — шум. На шаге t (t = T-1 … 0) генератор получает x_{t+1},
предсказывает x^_0, после чего x_t сэмплируется из апостериорного q(x_t | x_{t+1}, x^_0)."""
import math

import torch


def _bc(v, t, like):  # коэффициент для каждого элемента батча, расширенный до формы like
    return v[t].view(-1, *[1] * (like.dim() - 1)).to(like.dtype)


class Diffusion:
    def __init__(self, T: int = 8, device="cpu", s: float = 0.008):
        self.T = T
        f = torch.cos((torch.arange(T + 1) / T + s) / (1 + s) * math.pi / 2) ** 2
        abar = (f / f[0]).clamp(1e-5, 1.0)                    # ᾱ_0 = 1 … ᾱ_T ≈ 0
        alpha = torch.cat([torch.ones(1), abar[1:] / abar[:-1]])
        beta = 1 - alpha
        # коэффициенты апостериорного q(x_t | x_{t+1}, x_0) (стандартная формула DDPM)
        a1, ab1, ab0 = alpha[1:], abar[1:], abar[:-1]
        self.c0 = beta[1:] * ab0.sqrt() / (1 - ab1)           # при x_0
        self.c1 = (1 - ab0) * a1.sqrt() / (1 - ab1)           # при x_{t+1}
        self.std = (beta[1:] * (1 - ab0) / (1 - ab1)).sqrt()
        self.abar, self.alpha = abar, alpha
        for k in ("c0", "c1", "std", "abar", "alpha"):
            setattr(self, k, getattr(self, k).to(device))

    def q_pair(self, x0, t):
        """Реальные данные: зашумляем x0 и получаем пару (x_t, x_{t+1})."""
        xt = _bc(self.abar.sqrt(), t, x0) * x0 + _bc((1 - self.abar).sqrt(), t, x0) * torch.randn_like(x0)
        a = _bc(self.alpha, t + 1, x0)
        return xt, a.sqrt() * xt + (1 - a).sqrt() * torch.randn_like(x0)
```

```
    def posterior(self, x0, x_next, t):
        """x_t ~ q(x_t | x_{t+1}, x^_0). На последнем шаге (t=0) шум не добавляется."""
        mean = _bc(self.c0, t, x0) * x0 + _bc(self.c1, t, x0) * x_next
        return mean + _bc(self.std, t, x0) * (t > 0).float().view(-1, *[1] * (x0.dim() - 1)) *
        ↪  torch.randn_like(x0)

    @torch.no_grad()
    def sample(self, G, n: int, size: int, nz: int, device, z=None, logits=False):
        """Инференс: x_T ~ N(0, I), затем T шагов генератора. z фиксирован на всю траекторию.
        На последнем шаге (t=0) апостериорное равно x^_0, поэтому результат = tanh(логит).
        logits=True возвращает сам логит: для подбора порога без насыщения tanh."""
        x = torch.randn(n, 1, size, size, size, device=device)
        z = torch.randn(n, nz, device=device) if z is None else z
        for t in reversed(range(self.T)):
            tt = torch.full((n,), t, device=device, dtype=torch.long)
            last = logits and t == 0
            out = G(x, tt, z, logits=last)
            # posterior вызываем всегда: он тратит randn, и последовательность случайных чисел не
            ↪  меняется
            x = self.posterior(torch.tanh(out) if last else out, x, tt)
        return out if logits else x
```

A.3 src/dimexdam/models.py

```
"""3D-генератор (U-Net, условие на t и z) и 2D-дискриминатор (один на все плоскости, spectral
↪  norm).

Авторы используют тяжёлый NCSN++. Здесь облегчённая версия той же идеи: свёрточный U-Net,
который через AdaGN-модуляцию получает эмбеддинг шага t и латентного вектора z."""
import torch
import torch.nn as nn
import torch.nn.functional as F

SN = nn.utils.spectral_norm


class ResBlock3D(nn.Module):
    def __init__(self, cin, cout, emb):
        super().__init__()
        self.n1, self.c1 = nn.GroupNorm(8, cin), nn.Conv3d(cin, cout, 3, padding=1)
        self.n2, self.c2 = nn.GroupNorm(8, cout), nn.Conv3d(cout, cout, 3, padding=1)
        self.mod = nn.Linear(emb, 2 * cout)                    # AdaGN: scale и shift вычисляются
        ↪  из (t, z)
        self.skip = nn.Conv3d(cin, cout, 1) if cin != cout else nn.Identity()

    def forward(self, x, e):
        h = self.c1(F.silu(self.n1(x)))
        scale, shift = self.mod(e)[:, :, None, None, None].chunk(2, 1)
        h = self.c2(F.silu(self.n2(h) * (1 + scale) + shift))
        return h + self.skip(x)


class Generator3D(nn.Module):
    """G(x_{t+1}, t, z) -> x^_0 в диапазоне [-1, 1]. Полностью свёрточный, поэтому на инференсе
    можно генерировать объёмы любого размера, кратного 2^(len(ch)-1)."""

    def __init__(self, ch=(32, 64, 128, 128), nz=100, emb=128, T=8):
        super().__init__()
```

```
        self.t_emb = nn.Embedding(T, emb)
        self.z_mlp = nn.Sequential(nn.Linear(nz, emb), nn.SiLU(), nn.Linear(emb, emb), nn.SiLU(),
        ↪  nn.Linear(emb, emb))
        self.inp = nn.Conv3d(1, ch[0], 3, padding=1)
        self.enc = nn.ModuleList(ResBlock3D(ci, co, emb) for ci, co in zip((ch[0],) + ch[:-1],
        ↪  ch))
        self.mid = ResBlock3D(ch[-1], ch[-1], emb)
        self.dec = nn.ModuleList(ResBlock3D(ci + co, co, emb) for ci, co in zip(ch[:0:-1],
        ↪  ch[-2::-1]))
        self.out = nn.Sequential(nn.GroupNorm(8, ch[0]), nn.SiLU(), nn.Conv3d(ch[0], 1, 3,
        ↪  padding=1))

    def forward(self, x, t, z, logits=False):
        e = self.t_emb(t) + self.z_mlp(z)
        h, skips = self.inp(x), []
        for i, blk in enumerate(self.enc):                      # энкодер: 64 -> 32 -> 16 -> 8
            h = blk(F.avg_pool3d(h, 2) if i else h, e)
            skips.append(h)
        h = self.mid(h, e)
        for blk, s in zip(self.dec, reversed(skips[:-1])):      # декодер со skip-соединениями
            h = blk(torch.cat([F.interpolate(h, scale_factor=2), s], 1), e)
        out = self.out(h)
        return out if logits else torch.tanh(out)               # logits=True: выход до tanh (не
        ↪  насыщается)


class Discriminator2D(nn.Module):
    """D(x_t, x_{t+1}, t, plane): пара 2D-срезов, эмбеддинг шага и эмбеддинг плоскости.
    Spectral norm стоит на всех слоях. На выходе tanh(logit / τ), τ = 0.4 (раздел 3.3.1)."""

    def __init__(self, ch=64, emb=128, T=8, size=64, tau=0.4):
        super().__init__()
        self.tau = tau
        self.t_emb, self.p_emb = nn.Embedding(T, emb), nn.Embedding(3, emb)
        chs = [ch, ch * 2, ch * 4, ch * 8, ch * 8][: 1 + (size // 4).bit_length() - 1]  #
        ↪  downsample до 4×4
        self.convs = nn.ModuleList([SN(nn.Conv2d(2, chs[0], 3, 1, 1))] +
                                   [SN(nn.Conv2d(a, b, 4, 2, 1)) for a, b in zip(chs, chs[1:])])
        self.cond = nn.ModuleList(SN(nn.Linear(2 * emb, c)) for c in chs)  # условие добавляется
        ↪  в каждый слой
        self.head = SN(nn.Linear(chs[-1] * 16, 1))

    def forward(self, x_t, x_next, t, plane, return_raw=False):
        e = torch.cat([self.t_emb(t), self.p_emb(plane)], 1)
        h = torch.cat([x_t, x_next], 1)
        for conv, c in zip(self.convs, self.cond):
            h = F.leaky_relu(conv(h) + c(e)[:, :, None, None], 0.2)
        raw = self.head(h.flatten(1)).squeeze(1).clamp(-10, 10)   # logit clipping
        return raw if return_raw else torch.tanh(raw / self.tau)
```

A.4 src/dimexdam/train.py

```
"""Цикл обучения DimExDAM. Логика повторяет train_ddgan.py авторов: на каждом шаге t цепочки
сначала обновляется D, потом G, после чего цепочка отсоединяется (x = x_t.detach())."""
from dataclasses import asdict, dataclass

import torch
import torch.nn.functional as F
from tqdm.auto import tqdm
```

```python

from .data import SliceSampler, stratified_slices
from .diffusion import Diffusion
from .models import Discriminator2D, Generator3D


@dataclass
class Config:
    size: int = 64            # сторона генерируемого куба и 2D-кропа
    T: int = 8                # шагов диффузии
    nz: int = 100
    n_vol: int = 2            # 3D-объёмов на итерацию
    S: int = 8                # стратифицированных срезов на ось (итого 3*S*n_vol фейковых
    ↪  срезов)
    n_real: int = 32          # реальных кропов на шаг D
    lr_g: float = 2e-4        # TTUR: lr генератора больше, чем у D
    lr_d: float = 1e-5
    r1_gamma: float = 10.0    # lazy R1
    r1_every: int = 16
    g_clip: float = 3.0
    iters_per_epoch: int = 100
    amp: bool = True          # bf16 autocast


def blur_schedule(epoch: int):
    """Low-pass curriculum из статьи. Возвращает (sigma, вероятность размыть кроп)."""
    return (0.8, 0.65) if epoch < 4 else (0.4, 0.45) if epoch < 7 else (0.0, 0.0)


class Trainer:
    def __init__(self, slices, cfg: Config, device="cuda"):
        self.cfg, self.dev = cfg, device
        self.G = Generator3D(nz=cfg.nz, T=cfg.T).to(device)
        self.D = Discriminator2D(T=cfg.T, size=cfg.size).to(device)
        self.diff = Diffusion(cfg.T, device)
        self.real = SliceSampler(slices, cfg.size, device)
        self.optG = torch.optim.Adam(self.G.parameters(), cfg.lr_g, betas=(0.0, 0.9))
        self.optD = torch.optim.Adam(self.D.parameters(), cfg.lr_d, betas=(0.0, 0.9))
        self.step, self.hist = 0, {"lossD": [], "lossG": []}

    def _ac(self):
        return torch.autocast("cuda", torch.bfloat16, enabled=self.cfg.amp and self.dev ==
        ↪  "cuda")

    def train_iter(self):
        c, G, D = self.cfg, self.G, self.D
        blur = blur_schedule(self.step // c.iters_per_epoch)
        x = torch.randn(c.n_vol, 1, c.size, c.size, c.size, device=self.dev)   # x_T
        z = torch.randn(c.n_vol, c.nz, device=self.dev)
        lD = lG = 0.0
        for t in reversed(range(c.T)):
            tv = lambda n: torch.full((n,), t, device=self.dev, dtype=torch.long)
            with self._ac():                                    # 1) один шаг генератора по
            ↪  3D-цепочке
                x_t = self.diff.posterior(G(x, tv(c.n_vol), z), x, tv(c.n_vol))
            f_t, f_next, f_p = stratified_slices(x_t, x, c.S)

            # 2) шаг D: реальные пары (x_t, x_{t+1}) против срезов сгенерированного объёма
            D.requires_grad_(True)
            real, r_p = self.real(c.n_real, blur)
            r_t, r_next = self.diff.q_pair(real, tv(c.n_real))
```

```
            with self._ac():
                loss_d = F.softplus(-D(r_t, r_next, tv(c.n_real), r_p)).mean() + \
                         F.softplus(D(f_t.detach(), f_next, tv(len(f_p)), f_p)).mean()
            if self.step % c.r1_every == 0:                    # lazy R1 на реальных данных (в fp32)
                r_t = r_t.detach().requires_grad_(True)
                g, = torch.autograd.grad(D(r_t, r_next, tv(c.n_real), r_p).sum(), r_t, create_graph=True)
                loss_d = loss_d + c.r1_gamma / 2 * g.pow(2).flatten(1).sum(1).mean()
            self.optD.zero_grad(set_to_none=True); loss_d.backward(); self.optD.step()

            # 3) шаг G: non-saturating loss по тем же срезам
            D.requires_grad_(False)
            with self._ac():
                loss_g = F.softplus(-D(f_t, f_next, tv(len(f_p)), f_p)).mean()
            self.optG.zero_grad(set_to_none=True); loss_g.backward()
            torch.nn.utils.clip_grad_norm_(G.parameters(), c.g_clip)
            self.optG.step()

            x = x_t.detach().float()                           # переходим к следующему шагу цепочки
            lD += loss_d.item() / c.T; lG += loss_g.item() / c.T
        self.step += 1
        self.hist["lossD"].append(lD); self.hist["lossG"].append(lG)
        return lD, lG

    def fit(self, n_iters: int, ckpt=None, save_every=200):
        pbar = tqdm(range(n_iters))
        for i in pbar:
            lD, lG = self.train_iter()
            pbar.set_postfix(D=f"{lD:.3f}", G=f"{lG:.3f}", epoch=self.step // self.cfg.iters_per_epoch)
            if ckpt and (i + 1) % save_every == 0:
                self.save(ckpt)
        if ckpt:
            self.save(ckpt)

    def save(self, path):
        torch.save({"G": self.G.state_dict(), "D": self.D.state_dict(), "optG": self.optG.state_dict(),
                    "optD": self.optD.state_dict(), "cfg": asdict(self.cfg), "step": self.step,
                    "hist": self.hist}, path)

    def load(self, path):
        ck = torch.load(path, map_location=self.dev)
        for k in ("G", "D", "optG", "optD"):
            getattr(self, k).load_state_dict(ck[k])
        self.step, self.hist = ck["step"], ck["hist"]


def load_generator(path, device="cuda"):
    """Для инференса нужен только генератор и конфиг."""
    ck = torch.load(path, map_location=device)
    cfg = Config(**ck["cfg"])
    G = Generator3D(nz=cfg.nz, T=cfg.T).to(device).eval()
    G.load_state_dict(ck["G"])
    return G, cfg
```

A.5 src/dimexdam/metrics.py

```python
"""Простые дескрипторы для сравнения с ground truth: доля фазы и двухточечная функция S2(r)."""
import numpy as np


def phase_fraction(vol: np.ndarray, phase: int = 0) -> float:
    return float((vol == phase).mean())


def s2(vol: np.ndarray, axis: int, phase: int = 0, rmax: int = 32) -> np.ndarray:
    """S2(r) = P(обе точки на расстоянии r вдоль оси axis лежат в фазе phase).
    Считается через FFT, граница периодическая."""
    ind = (vol == phase).astype(np.float32)
    f = np.fft.rfft(ind, axis=axis)
    ac = np.fft.irfft(f * np.conj(f), n=ind.shape[axis], axis=axis) / ind.shape[axis]
    return ac.mean(axis=tuple(a for a in range(3) if a != axis))[:rmax]


def random_subvolumes(vol: np.ndarray, n: int, size: int, seed=0) -> list[np.ndarray]:
    """Случайные кубы size³ из ground-truth керна: эталон для сравнения со сгенерированными."""
    rng = np.random.default_rng(seed)
    out = []
    for _ in range(n):
        i, j, k = (rng.integers(0, s - size + 1) for s in vol.shape)
        out.append(vol[i:i + size, j:j + size, k:k + size])
    return out
```

A.6 scripts/generate_samples.py

```python
"""Обучает модель на вариант (если чекпойнта нет), генерирует сэмплы и сохраняет картинку со
↪  срезами.

    python scripts/generate_samples.py Variant_01 Variant_04 --n 8 --iters 3000
"""
import argparse
import json
import time
from pathlib import Path

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np
import torch

from dimexdam.data import load_mhd, ortho_slices
from dimexdam.diffusion import Diffusion
from dimexdam.metrics import phase_fraction, random_subvolumes
from dimexdam.train import Config, Trainer, load_generator

ROOT = Path(__file__).resolve().parents[1]


def gpu_mem(prefix):
    """Пиковая память GPU: allocated — тензоры PyTorch, reserved — примерно то, что видно в
    ↪  nvidia-smi."""
    return {f"{prefix}_peak_alloc_gb": round(torch.cuda.max_memory_allocated() / 1e9, 2),
            f"{prefix}_peak_reserved_gb": round(torch.cuda.max_memory_reserved() / 1e9, 2)}
```

```

p = argparse.ArgumentParser()
p.add_argument("variants", nargs="+")
p.add_argument("--n", type=int, default=8, help="сколько объёмов сгенерировать")
p.add_argument("--size", type=int, default=64)
p.add_argument("--iters", type=int, default=3000)
args = p.parse_args()

for var in args.variants:
    vol = load_mhd(ROOT / "data" / var / "model.mhd")
    ckpt = ROOT / "checkpoints" / f"{var}.pt"
    ckpt.parent.mkdir(exist_ok=True)
    out = ROOT / "outputs" / var; out.mkdir(parents=True, exist_ok=True)
    stats_path = out / "stats.json"
    stats = json.loads(stats_path.read_text()) if stats_path.exists() else {}
    if not ckpt.exists():                                   # обучаем на 3 центральных срезах
        torch.manual_seed(0); np.random.seed(0)
        torch.cuda.reset_peak_memory_stats(); t0 = time.time()
        Trainer(ortho_slices(vol), Config()).fit(args.iters, ckpt=ckpt)
        stats.update(train_iters=args.iters, train_min=round((time.time() - t0) / 60, 1),
        ↪  **gpu_mem("train"))

    G, cfg = load_generator(ckpt)
    diff = Diffusion(cfg.T, "cuda")
    torch.manual_seed(1)
    torch.cuda.synchronize(); torch.cuda.reset_peak_memory_stats(); t0 = time.time()
    gen = torch.cat([diff.sample(G, 4, args.size, cfg.nz, "cuda") for _ in range(-(-args.n //
    ↪  4))])[: args.n]
    torch.cuda.synchronize()
    stats.update(n_samples=args.n, size=args.size, sample_sec_per_rock=round((time.time() - t0) /
    ↪  args.n, 3),
                 **gpu_mem("sample"))
    gen = (gen[:, 0].cpu().numpy() > 0).astype(np.uint8)    # 0 = поры, 1 = твёрдая фаза
    np.save(out / "samples.npy", gen)
    stats_path.write_text(json.dumps(stats, indent=1))
    print(var, stats)

    # Картинка: первая строка — куб из керна, дальше первые 8 сэмплов. Колонки — центральные
    ↪  срезы XY/XZ/YZ.
    rows = [("керн", random_subvolumes(vol, 1, args.size)[0])] + [(f"сэмпл {i}", g) for i, g in
    ↪  enumerate(gen[:8])]
    fig, ax = plt.subplots(len(rows), 3, figsize=(6, 2 * len(rows)))
    for r, (name, v) in enumerate(rows):
        for a in range(3):
            ax[r, a].imshow(np.take(v, v.shape[a] // 2, axis=a), cmap="gray", vmin=0, vmax=1)
            ax[r, a].set_xticks([]); ax[r, a].set_yticks([])
        ax[r, 0].set_ylabel(f"{name}\nφ0={phase_fraction(v):.3f}", fontsize=8)
    for a, n in enumerate(["XY", "XZ", "YZ"]): ax[0, a].set_title(n)
    fig.suptitle(f"{var}: доля фазы 0 в керне {phase_fraction(vol):.3f}")
    fig.tight_layout(rect=(0, 0, 1, 0.98)); fig.savefig(out / "samples.png", dpi=110);
    ↪  plt.close(fig)
    print(var, "готово:", out / "samples.png")
```

A.7 scripts/calibrate_threshold.py

```

"""Порог бинаризации по пористости 2D-обучающих срезов (используется только 2D-информация).

1. Повторяет генерацию из generate_samples.py (тот же seed) и сохраняет «серые» объёмы.
```

```
2. Проверяет, что порог 0 даёт ровно тот же samples.npy, что был отправлен.
3. Подбирает один общий на вариант порог c: доля вокселей (grey < c) по всем сэмплам
   равна пористости трёх обучающих срезов. Порог один на все 20 объёмов, поэтому
   разброс между ними не подгоняется искусственно.

    python scripts/calibrate_threshold.py Variant_11 Variant_02
"""
import argparse
from pathlib import Path

import numpy as np
import torch

from dimexdam.data import load_mhd, ortho_slices
from dimexdam.diffusion import Diffusion
from dimexdam.train import load_generator

ROOT = Path(__file__).resolve().parents[1]
p = argparse.ArgumentParser()
p.add_argument("variants", nargs="+")
p.add_argument("--n", type=int, default=20)
p.add_argument("--size", type=int, default=64)
p.add_argument("--pad", type=int, default=0,
               help="генерировать (size+2*pad)³ и вырезать центр size³: убирает краевой артефакт (pad кратен 4)")
args = p.parse_args()
sfx = f"_crop{args.pad}" if args.pad else ""           # файлы с обрезкой не перезаписывают основные

for var in args.variants:
    out = ROOT / "outputs" / var
    out.mkdir(parents=True, exist_ok=True)
    G, cfg = load_generator(ROOT / "checkpoints" / f"{var}.pt")
    diff = Diffusion(cfg.T, "cuda")
    torch.manual_seed(1)                                # тот же seed, что в generate_samples.py
    L, q = args.size + 2 * args.pad, slice(args.pad, args.pad + args.size)
    logit = torch.cat([diff.sample(G, 4, L, cfg.nz, "cuda", logits=True) for _ in range(-(-args.n // 4))])
    logit = logit[: args.n, 0, q, q, q].float().cpu().numpy()   # (N, 64, 64, 64), выход до tanh
    grey = np.tanh(logit)                               # «серый» объём в [-1, 1], 99% вокселей насыщены
    np.save(out / f"samples_grey{sfx}.npy", grey.astype(np.float16))
    np.save(out / f"samples_logit{sfx}.npy", logit.astype(np.float16))

    # отправленный ранее файл: samples.npy или переименованный var_XX.npy (может лежать в другой папке)
    ref = [out / "samples.npy"] + sorted((ROOT / "outputs").rglob(f"var_{var[-2:]}.npy"))
    ref = next(f for f in ref if f.exists())
    old = np.load(ref)
    same = np.array_equal((grey > 0).astype(np.uint8), old)   # при --pad объёмы другие, будет False
    if args.pad:
        np.save(out / f"samples{sfx}.npy", (grey > 0).astype(np.uint8))

    # Целевая пористость: пиксели фазы 0 в трёх обучающих срезах (в них фаза 0 закодирована как -1)
    target = float(np.mean([(s < 0).float().mean() for s in ortho_slices(load_mhd(ROOT / "data" / var / "model.mhd"))]))
    # Порог по логиту: tanh монотонен, так что это тот же «серый» порог, но без потолка насыщения
    c = float(np.quantile(logit, target))                   # поры = logit < c
```

```
    cal = (logit >= c).astype(np.uint8)                    # 0 = поры, 1 = твёрдая фаза
    np.save(out / f"samples_cal{sfx}.npy", cal)

    base = (grey > 0).astype(np.uint8)
    por0, por1 = (base == 0).mean(axis=(1, 2, 3)), (cal == 0).mean(axis=(1, 2, 3))
    faces = lambda a: np.mean([np.take(a == 0, i, axis=k).mean() for k in (1, 2, 3) for i in (0, -1)])
    print(f"{var}: совпадает с {ref.relative_to(ROOT)} при пороге 0: {same} | цель (2D-срезы) {target:.3f} | порог по логиту {c:+.2f} (tanh = {np.tanh(c):+.4f})\n"
          f"   пористость при пороге 0: {por0.mean():.3f} ± {por0.std():.3f} -> после калибровки: {por1.mean():.3f} ± {por1.std():.3f}\n"
          f"   пористость на гранях: порог 0 {faces(base):.3f}, калибровка {faces(cal):.3f}"
          f" | доля «серых» вокселей (|grey| < 0.9): {(np.abs(grey) < 0.9).mean():.4f}")
```

A.8 scripts/view_volume.py

```
"""Просмотр 3D-объёма: сгенерированного (.npy) или исходного керна (.mhd).

    python scripts/view_volume.py outputs/Variant_01/samples.npy              # 3D-рендер сэмпла 0
    python scripts/view_volume.py outputs/Variant_01/samples.npy -i 3 --slices  # листать срезы
    python scripts/view_volume.py data/Variant_01/model.mhd --crop 64         # кусок керна для сравнения
    python scripts/view_volume.py outputs/Variant_01/samples.npy --mhd        # экспорт для ParaView

3D-рендер сохраняется как HTML и открывается в браузере (мышью крутить, колесом зум).
"""
import argparse
import gzip
from pathlib import Path

import numpy as np

from dimexdam.data import load_mhd

p = argparse.ArgumentParser()
p.add_argument("path", type=Path)
p.add_argument("-i", "--index", type=int, default=0, help="номер объёма в .npy")
p.add_argument("--phase", type=int, default=0, help="какую фазу рисовать поверхностью (0 = поры)")
p.add_argument("--crop", type=int, default=None, help="вырезать куб crop³ из начала объёма (для больших кернов)")
p.add_argument("--slices", action="store_true", help="вместо 3D листать срезы слайдером")
p.add_argument("--mhd", action="store_true", help="только сохранить .mhd/.raw.gz для ParaView")
args = p.parse_args()

# --- загрузка: .npy содержит (N, D, H, W), .mhd — один объём ---
vol = load_mhd(args.path) if args.path.suffix == ".mhd" else np.load(args.path)[args.index]
if args.crop:
    vol = vol[: args.crop, : args.crop, : args.crop]
name = f"{args.path.stem}_{args.index}" if args.path.suffix == ".npy" else "core"
out_dir = Path("outputs") / args.path.parent.name          # всё сохраняется в outputs/<вариант>/
out_dir.mkdir(parents=True, exist_ok=True)
print(f"{name}: форма {vol.shape}, доля фазы {args.phase} = {(vol == args.phase).mean():.3f}")

if args.mhd:  # --- экспорт в формат исходных данных ---
    out = out_dir / f"{name}.mhd"
    gzip.open(out.with_suffix(".raw.gz"), "wb").write(vol.astype(np.uint8).tobytes())
```

```
    D, H, W = vol.shape
    out.write_text("ObjectType = Image\nNDims = 3\nElementType = MET_UCHAR\nCompressedData = True\n"
                   f"DimSize = {W} {H} {D}\nElementSpacing = 1 1 1\nElementDataFile = {out.stem}.raw.gz\n")
    print("сохранено:", out)

elif args.slices:  # --- листалка срезов по трём осям ---
    import matplotlib.pyplot as plt
    from matplotlib.widgets import Slider
    fig, ax = plt.subplots(1, 3, figsize=(12, 4.5))
    ims = [a.imshow(np.take(vol, vol.shape[k] // 2, axis=k), cmap="gray", vmin=0, vmax=1) for k, a in enumerate(ax)]
    for a, t in zip(ax, ["XY", "XZ", "YZ"]): a.set_title(t); a.axis("off")
    s = Slider(fig.add_axes([0.2, 0.05, 0.6, 0.03]), "срез", 0, min(vol.shape) - 1, valinit=min(vol.shape) // 2, valstep=1)
    s.on_changed(lambda v: [im.set_data(np.take(vol, int(v), axis=k)) for k, im in enumerate(ims)] and fig.canvas.draw_idle())
    plt.show()

else:  # --- 3D: поверхность выбранной фазы (marching cubes) в plotly ---
    import plotly.graph_objects as go
    from skimage.measure import marching_cubes
    # Обкладываем объём рамкой из другой фазы, чтобы поверхность закрывалась на гранях куба
    mask = np.pad((vol == args.phase).astype(np.float32), 1)
    v, f, _, _ = marching_cubes(mask, 0.5)
    fig = go.Figure(go.Mesh3d(x=v[:, 2], y=v[:, 1], z=v[:, 0], i=f[:, 0], j=f[:, 1], k=f[:, 2],
                              color="steelblue", flatshading=True,
                              lighting=dict(ambient=0.4, diffuse=0.8), opacity=1.0))
    fig.update_layout(title=f"{name}: фаза {args.phase}", scene=dict(aspectmode="data"), margin=dict(l=0, r=0, t=40, b=0))
    out = out_dir / f"{name}_3d.html"
    fig.write_html(out, auto_open=True)
    print("3D-рендер:", out)
```